Arrays

❖ Types of Arrays

• Single Dimensional Array

A single-dimensional array is the simplest type of array that stores elements in a linear sequence. It is also known as a one-dimensional array, and each element in the array can be accessed using a single index. The index ranges from 0 to the length of the array minus one.

```
data_type array_name[array_size];

//Example

int evenNumbers[5];
```

data_type : is a type of data for each block of an array.
array_name : is the array's name, which we can use to refer to it.
array_size : is the number of memory array blocks will have.

```
data_type array_name[array_size] = {element1, element2, element3,....., elementN};

//Example

int evenNumbers[5] = {0, 2, 4, 6, 8};
```

• Two-Dimensional Array

A multidimensional array with two dimensions is known as a two-dimensional array. A two-dimensional array is an array of arrays, where each element in the array is itself an array. It is also known as a matrix, and can be used to represent tables, grids, or other two-dimensional structures.

```
data_type array_name[sizeof_1st_dimension][sizeof_2nd_dimension];

//Example

int array2d[5][10];
```

- **Multi-Dimensional Array**

A multi-dimensional array is an array that stores elements in a two-dimensional or higher-dimensional structure. It is also known as a two-dimensional or higher-dimensional array, and each element in the array can be accessed using multiple indices. Multi-dimensional arrays can be of different dimensions, including two-dimensional, three-dimensional, or higher-dimensional arrays.

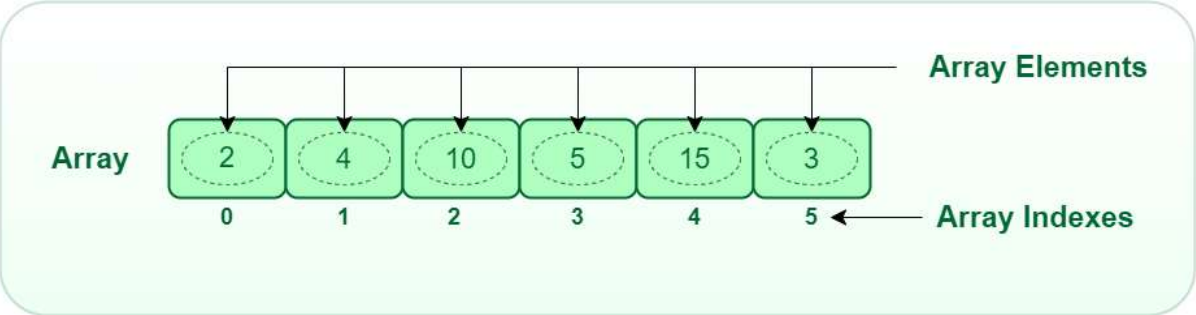
```
data_type
array_name[sizeof_1st_dimension][sizeof_2nd_dimension].....[sizeof_nth_dimension];


//Example


int array[5][4][6][7];
```

❖ **Representation of Array in Memory**

The representation of an array can be defined by its declaration.
A declaration means allocating memory for an array of a given size.



int arr[5]; // This array will store integer type element
char arr[10]; // This array will store char type element
float arr[20]; // This array will store float type element

❖ Operations on Arrays in Data Structures

1. **Traversal:** Array traversal refers to the process of accessing each element of an array in a sequential order from the beginning to the end or from the end to the beginning.
2. **Insertion:** Insertion is the process of adding a new element to an array. Depending on the application, the new element can be inserted at the beginning, end, or any other location within the array. When inserting an element, the other elements in the array may need to be shifted to accommodate the new element.
3. **Deletion:** Deletion is the process of removing an existing element from an array. Depending on the application, the element to be deleted can be at any location within the array. When deleting an element, the other elements in the array may need to be shifted to fill the gap left by the deleted element.
4. **Search:** Search is the process of finding a specific element within an array. This is done by comparing the target element with each element in the array until a match is found. There are various search algorithms that can be used, such as linear search and binary search.
5. **Sorting:** Sorting is the process of arranging the elements in an array in a specific order. The most common sorting algorithms are bubble sort, insertion sort, selection sort, merge sort, and quicksort. Sorting an array can help with searching and other operations, as it allows for efficient data processing.
6. **Access:** Access is the process of retrieving the value of an element from an array. This is done by specifying the index of the desired element within the array. Array access is fast and efficient, as it does not require traversal of the entire array.